

Course Code	Course Title	L	R	P	Cr
	AI-Assisted Problem Solving Using Python Programming				
Program Core					

Day 10 Session: GUI Development & Database Connectivity

Day 9

- Tkinter GUI Development
- Event Handling
- SQLite Database
- CRUD Operations
- AI-Assisted Desktop Application Development

Topics Covered

- Tkinter GUI Development
- Event Handling
- SQLite Database
- CRUD Operations
- AI-Assisted Desktop Application Development

Python GUI – tkinter:

- Tkinter is Python's standard GUI library.
- Python offers multiple options for developing GUI (Graphical User Interface).
- Out of all the GUI methods, tkinter is the most commonly used method.
- It is a standard Python interface to the Tk GUI toolkit shipped with Python.
- Python with tkinter is the fastest and easiest way to create the GUI applications.
- Creating a GUI using tkinter is an easy task.

To create a tkinter app:

- Importing the module – tkinter
- Create the main window (container)
- Add any number of widgets to the main window
- Apply the event Trigger on the widgets.

There are two main methods used which the user needs to remember while creating the Python application with GUI.

1. Tk(screenName=None, baseName=None, className='Tk', useTk=1):

To create a main window, tkinter offers a method

The basic code used to create the main window of the application is: `m=tkinter.Tk()` where `m` is the name of the main window object

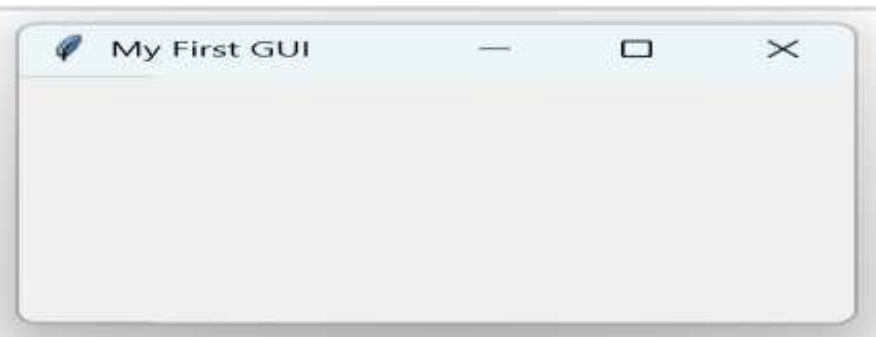
2. mainloop(): There is a method known by the name `mainloop()` is used when your application is ready to run.

`mainloop()` is an infinite loop used to run the application, wait for an event to occur and process the event as long as the window is not closed. `m.mainloop()`

Creating First Window

```
#Importing Package
from tkinter import *

#Creating object for class Tk()
w = Tk()
w.title("My First GUI")
w.geometry("400x300")
w.mainloop()
```



Common Tkinter Widgets

Widget	Purpose	Syntax	Example
Tk()	Creates the main application window	<code>root = tk.Tk()</code>	<code>root = tk.Tk()</code>
Label	Displays text or images	<code>tk.Label(parent, text="Text")</code>	<code>label = tk.Label(root, text="Welcome")</code>
Button	Executes a function when clicked	<code>tk.Button(parent, text="Click", command=function)</code>	<code>button = tk.Button(root, text="Submit", command=submit)</code>
Entry	Accepts single-line text input	<code>tk.Entry(parent)</code>	<code>entry = tk.Entry(root)</code>
Text	Accepts multi-line text input	<code>tk.Text(parent, height=h, width=w)</code>	<code>text = tk.Text(root, height=5, width=30)</code>
Frame	Groups related widgets	<code>tk.Frame(parent)</code>	<code>frame = tk.Frame(root)</code>
LabelFrame	Creates a labeled container	<code>tk.LabelFrame(parent, text="Title")</code>	<code>lf = tk.LabelFrame(root, text="Student Details")</code>
Canvas	Used for drawing graphics	<code>tk.Canvas(parent, width=w, height=h)</code>	<code>canvas = tk.Canvas(root, width=300, height=200)</code>
Checkbutton	Creates a checkbox	<code>tk.Checkbutton(parent, text="Text", variable=var)</code>	<code>check = tk.Checkbutton(root, text="Python", variable=var)</code>

Common Tkinter Widgets

Widget	Purpose	Syntax	Example
Radiobutton	Creates a radio button	<code>tk.Radiobutton(parent, text="Option", variable=var, value=v)</code>	<code>rb = tk.Radiobutton(root, text="Male", variable=gender, value="Male")</code>
Listbox	Displays a list of items	<code>tk.Listbox(parent)</code>	<code>listbox = tk.Listbox(root)</code>
Scrollbar	Adds scrolling functionality	<code>tk.Scrollbar(parent)</code>	<code>scroll = tk.Scrollbar(root)</code>
Scale	Creates a slider	<code>tk.Scale(parent, from_=min, to=max)</code>	<code>scale = tk.Scale(root, from_=0, to=100)</code>
Spinbox	Allows selecting values using arrows	<code>tk.Spinbox(parent, from_=min, to=max)</code>	<code>spin = tk.Spinbox(root, from_=1, to=10)</code>
Menu	Creates a menu bar	<code>tk.Menu(parent)</code>	<code>menu = tk.Menu(root)</code>
Menubutton	Displays a drop-down menu	<code>tk.Menubutton(parent, text="Menu")</code>	<code>mb = tk.Menubutton(root, text="Options")</code>
Message	Displays long formatted text	<code>tk.Message(parent, text="Message")</code>	<code>msg = tk.Message(root, text="Welcome to Tkinter")</code>
PanedWindow	Creates resizable panes	<code>tk.PanedWindow(parent)</code>	<code>pw = tk.PanedWindow(root)</code>
Toplevel	Creates an additional window	<code>tk.Toplevel(parent)</code>	<code>new_window = tk.Toplevel(root)</code>

Common Geometry Managers

Method	Purpose	Syntax	Example
pack()	Arranges widgets vertically or horizontally	<code>widget.pack()</code>	<code>label.pack()</code>
grid()	Arranges widgets in rows and columns	<code>widget.grid(row=r, column=c)</code>	<code>entry.grid(row=0, column=1)</code>
place()	Places widgets at specific coordinates	<code>widget.place(x=x, y=y)</code>	<code>button.place(x=100, y=50)</code>

Common Window Methods

Method	Purpose	Syntax
<code>title()</code>	Sets the window title	<code>root.title("Student App")</code>
<code>geometry()</code>	Sets the window size	<code>root.geometry("500x400")</code>
<code>configure()</code>	Changes window properties	<code>root.configure(bg="lightblue")</code>
<code>resizable()</code>	Enables/disables window resizing	<code>root.resizable(False, False)</code>
<code>iconbitmap()</code>	Sets the window icon	<code>root.iconbitmap("icon.ico")</code>
<code>mainloop()</code>	Starts the GUI event loop	<code>root.mainloop()</code>

Common Widget Methods

Method	Purpose	Example
<code>get()</code>	Retrieves widget data	<code>entry.get()</code>
<code>insert()</code>	Inserts text	<code>entry.insert(0, "Python")</code>
<code>delete()</code>	Deletes content	<code>entry.delete(0, tk.END)</code>
<code>config()</code> / <code>configure()</code>	Modifies widget properties	<code>label.config(text="Success")</code>
<code>bind()</code>	Binds an event to a function	<code>button.bind("<Button-1>", click)</code>
<code>destroy()</code>	Removes a widget or closes a window	<code>root.destroy()</code>
<code>focus()</code>	Sets keyboard focus	<code>entry.focus()</code>
<code>pack_forget()</code>	Hides a packed widget	<code>label.pack_forget()</code>
<code>grid_forget()</code>	Hides a gridded widget	<code>button.grid_forget()</code>

tkinter also offers access to the geometric configuration of the widgets which can organize the widgets in the parent windows. There are mainly three geometry manager classes class.

- **pack() method:** It organizes the widgets in blocks before placing in the parent widget.
- **grid() method:** It organizes the widgets in grid (table-like structure) before placing in the parent widget.
- **place() method:** It organizes the widgets by placing them on specific positions directed by the programmer.

Label Widget

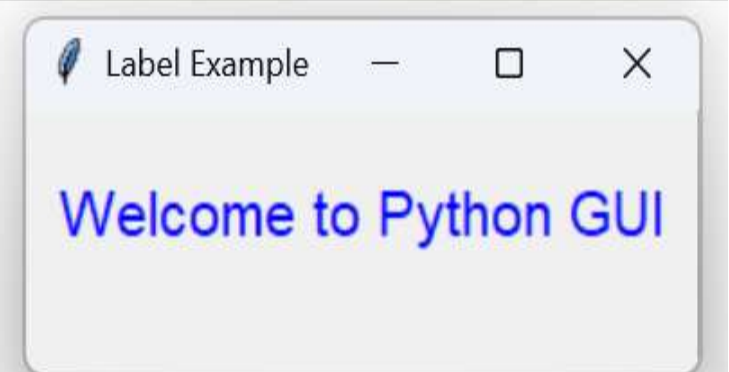
What is a Label Widget?

The **Label** widget is used to display **text, numbers, or images** in a Tkinter window. It is a non-editable widget, meaning users can view the information but cannot modify it. Labels are commonly used to provide instructions, headings, captions, or display results in a GUI application.

Features

- Displays static text or images.
- Supports customization such as font, color, background, and alignment.
- Can be updated dynamically using the config() method.

```
#Importing Package
from tkinter import *
#Creating object for class Tk()
w = Tk()
w.title("My First GUI")
w.geometry("400x300")
#Label Widget
w.title("Label Example")
label = Label(w, text="Welcome to Python GUI", font=("Arial", 16), fg="blue")
label.pack(pady=20)
w.mainloop()
```



Entry Widget

The **Entry** widget is used to accept **single-line text input** from the user. It is commonly used in forms such as login pages, registration forms, search boxes, and data entry applications.

Syntax: `w=Entry(master, option=value)`

`master` is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas.

Some of them are listed below.

bd: to set the border width in pixels.

bg: to set the normal background color.

cursor: to set the cursor used.

command: to call a function.

highlightcolor: to set the color shown in the focus highlight.

width: to set the width of the button.

height: to set the height of the button.

```
#Importing Package
from tkinter import *
#Creating object for class Tk()
w = Tk()
w.title("My First GUI")
w.geometry("400x300")
#Entry Widget
label = Label(w, text="Enter Your Name:", font=("Arial", 16), fg="blue")
label.pack()
entry = Entry(w, width=30)
entry.pack()
w.mainloop()
```



Entry Widget

```
from tkinter import *  
master = Tk()  
Label(master, text='First Name').grid(row=0)  
Label(master, text='Last Name').grid(row=1)  
e1 = Entry(master)  
e2 = Entry(master)  
e1.grid(row=0, column=1)  
e2.grid(row=1, column=1)  
mainloop()
```



1. Button: To add a button in your application, this widget is used.
The general syntax is:

`w=Button(master, option=value)`

master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the Buttons.
Number of options can be passed as parameters separated by commas.

Some of them are listed below.

activebackground: to set the background color when button is under the cursor.

activeforeground: to set the foreground color when button is under the cursor.

bg: to set the normal background color.

command: to call a function.

font: to set the font on the button label.

image: to set the image on the button.

width: to set the width of the button.

height: to set the height of the button.

Button Widget Example

```
def click():  
    print("Button clicked")  
  
button=tk.Button(root,text="Click",command=click)  
button.pack()
```

2. Canvas: It is used to draw pictures and other complex layout like graphics, text and widgets.

The general syntax is: `w = Canvas(master, option=value)`

There are number of options which are used to change the format of the widget.

Number of options can be passed as parameters separated by commas.

Some of them are listed below.

bd: to set the border width in pixels.

bg: to set the normal background color.

cursor: to set the cursor used in the canvas.

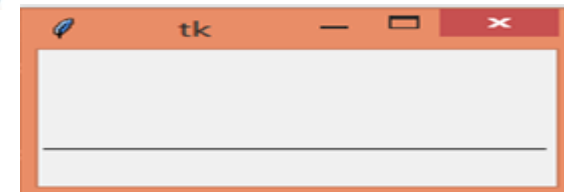
highlightcolor: to set the color shown in the focus highlight.

width: to set the width of the widget.

height: to set the height of the widget.

Canvas Widget Example

```
#Canvas
from tkinter import *
master = Tk()
w = Canvas(master, width=200, height=100)
w.pack()
canvas_height=150
canvas_width=200
y = int(canvas_height / 2)
w.create_line(0, y, canvas_width, y)
mainloop()
```



3. CheckButton: To select any number of options by displaying a number of options to a user as toggle buttons.

The general syntax is: `w = CheckButton(master, option=value)`

There are number of options which are used to change the format of this widget. Number of options can be passed as parameters separated by commas.

Some of them are listed below.

Title: To set the title of the widget.

activebackground: to set the background color when widget is under the cursor.

activeforeground: to set the foreground color when widget is under the cursor.

bg: to set the normal background

Secret Code:

command: to call a function.

font: to set the font on the button label.

image: to set the image on the widget.

```
#CheckButton
```

```
from tkinter import *
```

```
master = Tk()
```

```
var1 = IntVar()
```

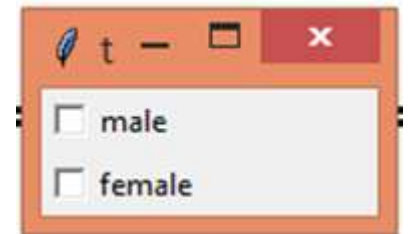
```
Checkbutton(master, text='male', variable=var1).grid(row=0, sticky=W)
```

```
var2 = IntVar()
```

```
Checkbutton(master, text='female', variable=var2).grid(row=1, sticky=W)
```

```
mainloop()
```

CheckButton Widget Example



4. Frame:

It acts as a container to hold the widgets. It is used for grouping and organizing the widgets.

The general syntax is: `w = Frame(master, option=value)`

- master is the parameter used to represent the parent window.
- There are number of options which are used to change the format of the widget.
- Number of options can be passed as parameters separated by commas.

Some of them are listed below.

highlightcolor: To set the color of the focus highlight when widget has to be focused.

bd: to set the border width in pixels.

bg: to set the normal background color.

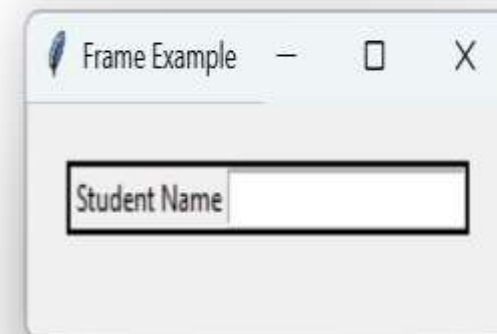
cursor: to set the cursor used.

width: to set the width of the widget.

height: to set the height of the widget.

Frame Widget Example

```
from tkinter import *
root = Tk()
root.title("Frame Example")
root.geometry("300x200")
frame = Frame(root, bg="lightgray", bd=2, relief="solid")
frame.pack(padx=20, pady=20)
label = Label(frame, text="Student Name")
label.pack( side = LEFT )
entry = Entry(frame)
entry.pack( side = RIGHT )
root.mainloop()
```



4. Frame Example:

```
#Frame
from tkinter import *
root = Tk()
frame = Frame(root)
frame.pack()
bottomframe = Frame(root)
bottomframe.pack( side = BOTTOM )
redbutton = Button(frame, text = 'Red', fg = 'red')
redbutton.pack( side = LEFT)
greenbutton = Button(frame, text = 'Brown', fg = 'brown')
greenbutton.pack( side = LEFT )
bluebutton = Button(frame, text = 'Blue', fg = 'blue')
bluebutton.pack( side = LEFT )
blackbutton = Button(bottomframe, text = 'Black', fg = 'black')
blackbutton.pack( side = BOTTOM)
root.mainloop()
```



Displaying Images:

To place an image into a Tkinter frame, you can follow the steps given below

Steps –

- Import the required libraries and create an instance of tkinter frame.
- To open an image and place it inside the frame, we will use the [Pillow \(PIL\) library](#).
- Set the size of the frame using **geometry** method.
- Create a frame and specify its height and width.
- Place the frame at the center of the window using **place()** method with **anchor='center'**.
- Open an image using **ImageTk.PhotoImage(Image.open("image"))**
- Next, create a **label** object inside the frame and pass the **image** inside the **label**.
- Finally, run the **mainloop** of the application window

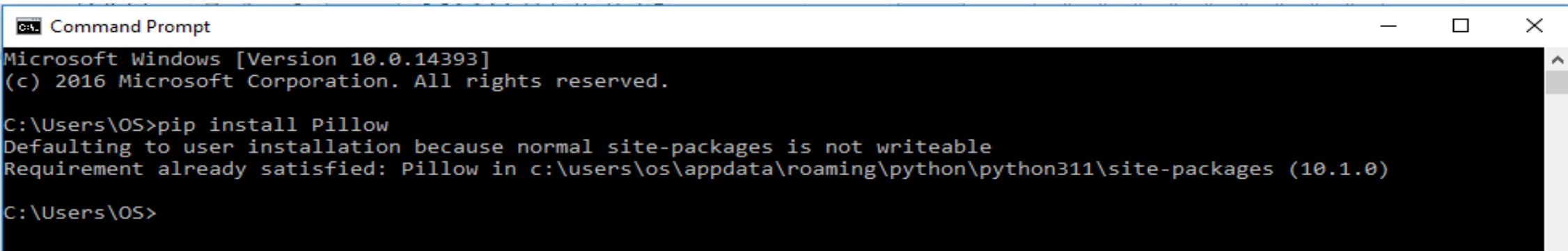
Displaying Images:

- To add an image to the window or Frame, first install package Pillow
- To Install Package 'Pillow' open command prompt and type: pip install Pillow
- Check that already pip install while installing IDLE

Steps to install Pillow

- Open Command Prompt (Type cmd in search)
- type the Command : pip install Pillow

After Sucessful Installation: if we want to add an image import a module
from PIL import ImageTk,Image



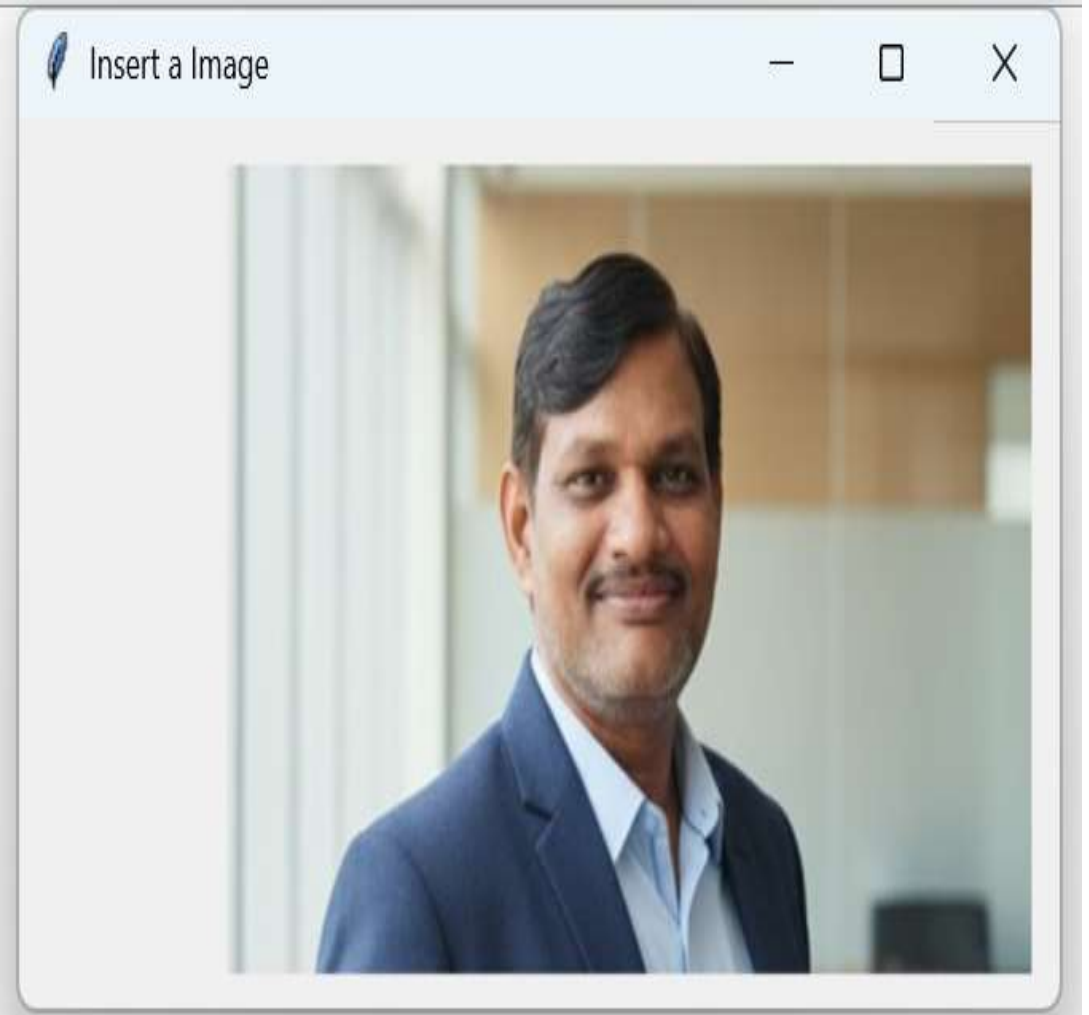
```
Command Prompt
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\OS>pip install Pillow
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: Pillow in c:\users\os\appdata\roaming\python\python311\site-packages (10.1.0)

C:\Users\OS>
```

Displaying Images: Example

```
from tkinter import *
from tkinter.ttk import *
from tkinter.messagebox import *
from PIL import ImageTk, Image
w = Tk()
w.geometry("600x400")
w.title("Insert a Image")
img1 = ImageTk.PhotoImage(Image.open("bvk.jpeg"))
lb1 = Label(w, image=img1)
lb1.grid(row=0, column=0)
lb1.pack(side=RIGHT, padx=10, pady=10)
w.mainloop()
```



Generating Colours:

Tkinter represents colors with strings. There are two general ways to specify colors in Tkinter – You can use a string specifying the proportion of red, green and blue in hexadecimal digits.

For example, "#fff" is white, "#000000" is black, "#000fff000" is pure green, and "#00ffff" is pure cyan (green plus blue).

You can also use any locally defined standard color name. The colors "white", "black", "red", "green", "blue", "cyan", "yellow", and "magenta" will always be available.

Color options

The common color options are

activebackground – Background color for the widget when the widget is active.

activeforeground – Foreground color for the widget when the widget is active.

background – Background color for the widget. This can also be represented as bg.

disabledforeground – Foreground color for the widget when the widget is disabled.

foreground – Foreground color for the widget. This can also be represented as fg.

highlightbackground – Background color of the highlight region when the widget has focus.

highlightcolor – Foreground color of the highlight region when the widget has focus.

selectbackground – Background color for the selected items of the widget.

selectforeground – Foreground color for the selected items of the widget.

Generating Colours Example:

```
#Generating Colors
from tkinter import *
w = Tk()
w.geometry("600x400")
w.title("Graphics Colors")
#Two Types of Color Formats
'''1. Directly with Color Names: , green,yellow etc..
2. Hexadecimal Color Format - RRGGBB
0-9 and A-F
RR - (0-F0-F) GG - (0-F0-F) BB - (0-F0-F)
#Example = '#FF0090'''
#Background Color = configure(background)
#w.configure(background='green')
#w.configure(bg='green')
#w['bg'] = 'orange'
#foreground - fg
#background - bg
btn1 = Button(w,text="Click Here",bg="pink",fg="orange")
btn1.pack(side = LEFT, padx = 20, pady = 20)
#Hexadecimal
#w.configure(background='#FFFFFF')
#w.configure(background='#000000')
#w.configure(background='#FF0000')
#w.configure(background='#00FF00')
#w.configure(background='#0000FF')
#w.configure(background='#091245')
w.configure(background='#EE00EE')
btn2 = Button(w,text="Submit",bg="#00FF10",fg="#123456")
btn2.pack(side = LEFT, padx = 10, pady = 10)
w.mainloop()
```



Task-I

Develop a GUI Application as shown below:

 Student Details

STUDENT INFORMATION

Name of The Student:

Roll No:

Department: ▼

Subject Name:

APTT
 PSP
 MATHS
 PHY

Date of Birth: ▼ ▼ ▼

Gender: ☒ Male ☐ Female

Email Id:

Address:

Languages Known: ☒ Telugu ☒ Hindi ☒ English

Task-II

Design a GUI containing two **Entry** widgets to accept the student's **Name** and **Roll Number**, along with a **Click Here** button. When the button is clicked, the program should display the message "**Welcome to SR University**" and print the entered **Name** and **Roll Number**.

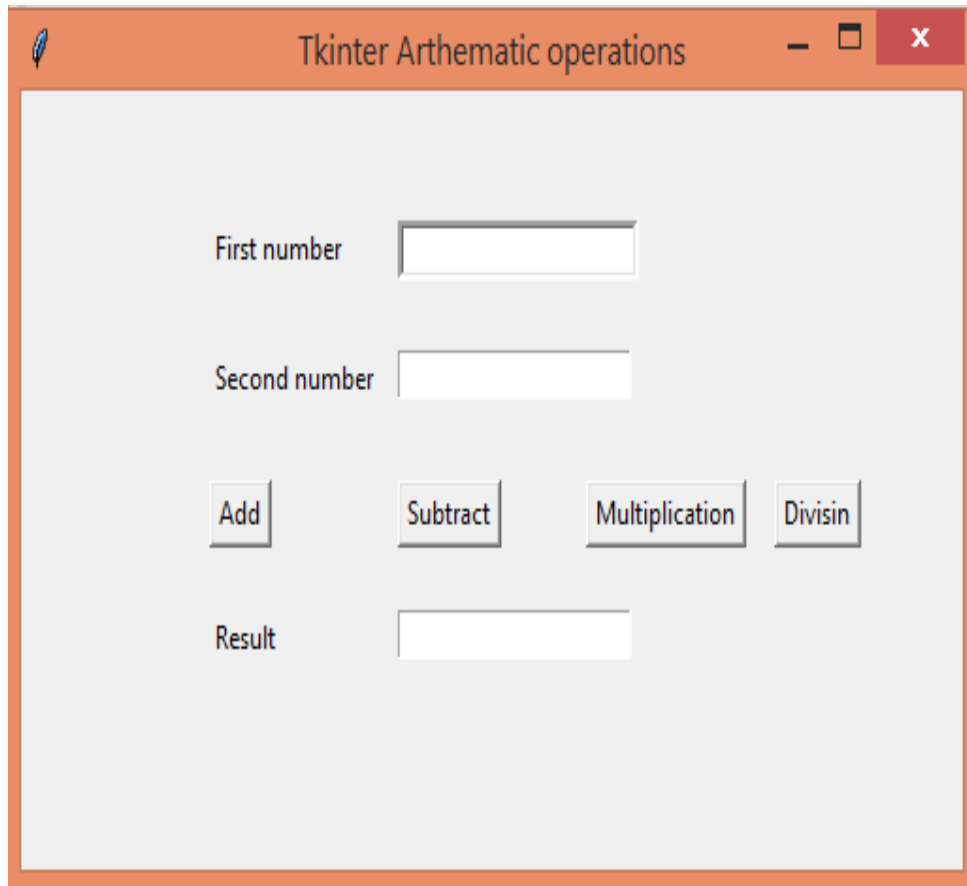
 Basic Student Details: — □ ×

Enter Name:

Enter Rollno:

Task-III

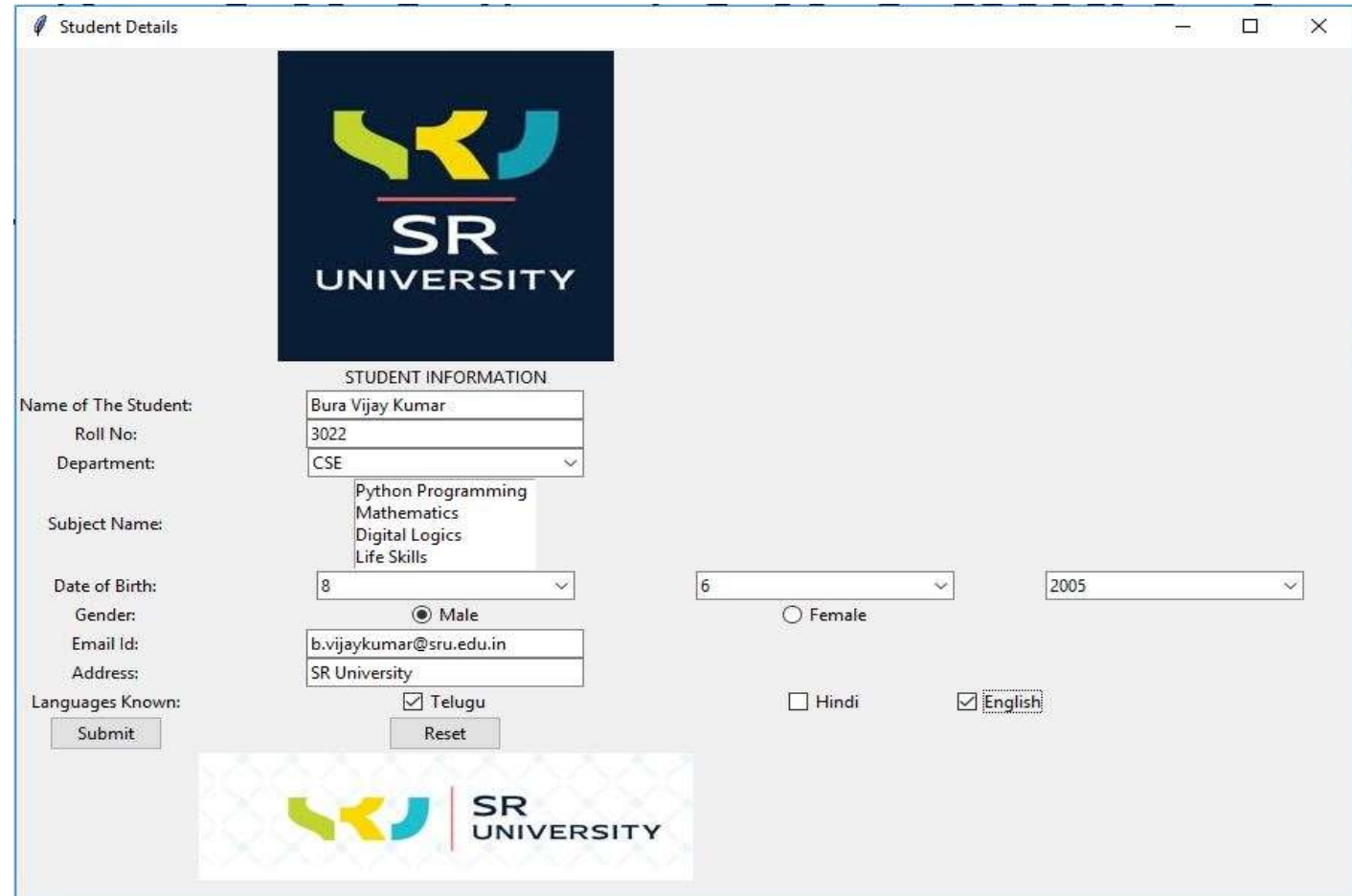
Develop a GUI Application to perform all arithmetic Operations



The screenshot shows a Tkinter window titled "Tkinter Arithmetic operations". It contains two input fields for "First number" and "Second number". Below these are four buttons: "Add", "Subtract", "Multiplication", and "Divisin" (misspelled). At the bottom, there is a "Result" label and an empty output field.

Task-IV

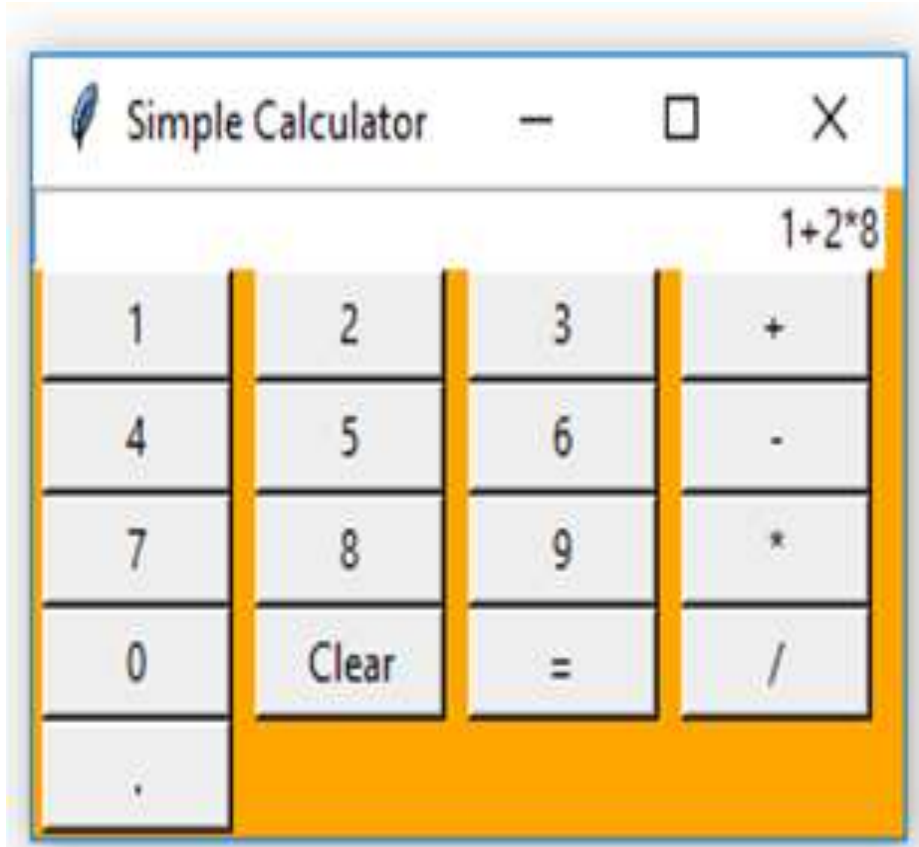
Develop a GUI Application as shown below



The screenshot shows a "Student Details" GUI window. It features the SR University logo at the top. Below the logo is a section titled "STUDENT INFORMATION" containing several form fields: "Name of The Student:" (Bura Vijay Kumar), "Roll No:" (3022), "Department:" (CSE), "Subject Name:" (a dropdown menu with options: Python Programming, Mathematics, Digital Logics, Life Skills), "Date of Birth:" (8), "Gender:" (radio buttons for Male and Female, with Male selected), "Email Id:" (b.vijaykumar@sru.edu.in), and "Address:" (SR University). There are also checkboxes for "Languages Known:" (Telugu and English, both checked). At the bottom, there are "Submit" and "Reset" buttons. The SR University logo is also displayed at the bottom of the window.

Task-V

Develop a GUI Application Calculator as shown below and perform the operations



Task-VI

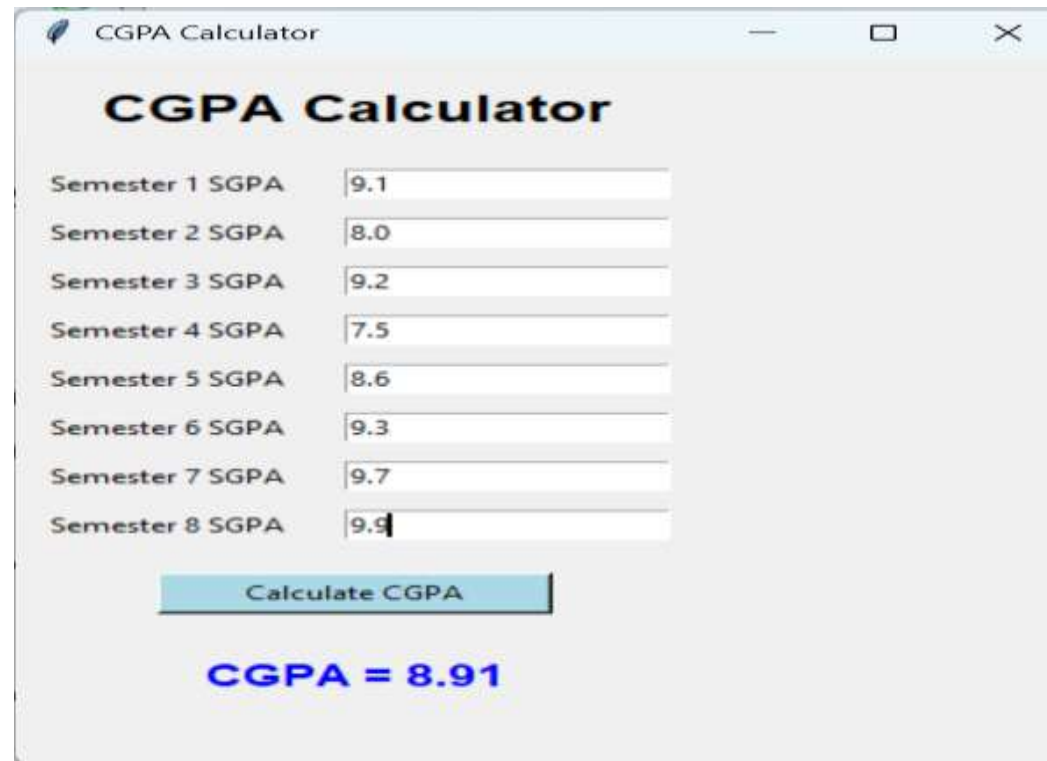
Design a GUI application that accepts the **SGPA** of **eight semesters** from the user.

When the **Calculate CGPA** button is clicked, the application should:
Read the SGPA values entered by the user.

Calculate the CGPA using the formula:

$$CGPA = \frac{SGPA_1 + SGPA_2 + \dots + SGPA_8}{8}$$

Display the calculated CGPA (rounded to two decimal places).



Python SQLite3

- **SQLite3** is a lightweight, serverless, self-contained relational database management system (RDBMS) that comes built into Python.
- It stores the entire database in a single file, making it ideal for desktop applications, projects, and educational purposes.

Features of SQLite3

- Lightweight and Fast
- Serverless Database
- Zero Configuration
- Cross-Platform
- Embedded Database Engine
- Open Source
- Stores data in a single .db file
- Supports SQL (Structured Query Language)

Advantages of SQLite3

- No separate database server is required.
- Easy to install (comes with Python).
- Portable database file.
- Fast for small and medium-sized applications.
- Supports transactions and indexing.
- Ideal for Tkinter desktop applications.

SQLite3 Data types:

SQLite uses a more general dynamic type system. In SQLite, the datatype of a value is associated with the value itself, not with its container

Types are:

- NULL**: The value is a NULL value.
- INTEGER**: The value is a signed integer, stored in 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value
- REAL**: The value is a floating point value, stored as an 8-byte IEEE floating point number.
- TEXT**. The value is a text string, stored using the database encoding (UTF-8, UTF-16BE or UTF-16LE). (default max 1,000,000,000 chars)
- BLOB**. The value is a blob of data, stored exactly as it was input.

SQLite Data Type	Description	Example
INTEGER	Whole Numbers	101
REAL	Decimal Numbers	8.75
TEXT	String Values	"Rahul"
BLOB	Binary Data	Images
NULL	Empty Value	NULL

SQLite3 Commands Used in Python:

Python SQLite3 Statement	Purpose	Example
<code>sqlite3.connect()</code>	Connects to or creates a database	<code>conn = sqlite3.connect("student.db")</code>
<code>conn.cursor()</code>	Creates a cursor object	<code>cursor = conn.cursor()</code>
<code>cursor.execute()</code>	Executes a SQL statement	<code>cursor.execute("SELECT * FROM Student")</code>
<code>cursor.executemany()</code>	Executes multiple SQL statements	<code>cursor.executemany(query, data)</code>
<code>cursor.fetchone()</code>	Retrieves one record	<code>row = cursor.fetchone()</code>
<code>cursor.fetchmany(n)</code>	Retrieves <i>n</i> records	<code>rows = cursor.fetchmany(5)</code>
<code>cursor.fetchall()</code>	Retrieves all records	<code>rows = cursor.fetchall()</code>
<code>conn.commit()</code>	Saves changes permanently	<code>conn.commit()</code>
<code>conn.rollback()</code>	Undoes uncommitted changes	<code>conn.rollback()</code>
<code>conn.close()</code>	Closes the database connection	<code>conn.close()</code>

SQLite3 Commands Used in Python:

SQL Command	Purpose	Syntax	Example
CREATE TABLE	Creates a new table	CREATE TABLE table_name(column1 datatype, column2 datatype, ...);	CREATE TABLE Student(id INTEGER PRIMARY KEY, name TEXT, cgpa REAL);
INSERT INTO	Inserts a new record into a table	INSERT INTO table_name(column1, column2, ...) VALUES(value1, value2, ...);	INSERT INTO Student(name, cgpa) VALUES('Rahul', 8.90);
SELECT	Retrieves all records from a table	SELECT * FROM table_name;	SELECT * FROM Student;
SELECT Specific Columns	Retrieves selected columns	SELECT column1, column2 FROM table_name;	SELECT name, cgpa FROM Student;
WHERE	Retrieves records based on a condition	SELECT * FROM table_name WHERE condition;	SELECT * FROM Student WHERE cgpa > 8.5;
UPDATE	Updates existing records	UPDATE table_name SET column=value WHERE condition;	UPDATE Student SET cgpa=9.25 WHERE id=1;

SQLite3 Commands Used in Python:

SQL Command	Purpose	Syntax	Example
DELETE	Deletes records from a table	DELETE FROM table_name WHERE condition;	DELETE FROM Student WHERE id=1;
DROP TABLE	Deletes an entire table	DROP TABLE table_name;	DROP TABLE Student;
ALTER TABLE	Adds a new column to a table	ALTER TABLE table_name ADD COLUMN column_name datatype;	ALTER TABLE Student ADD COLUMN branch TEXT;
ORDER BY	Sorts records in ascending or descending order	SELECT * FROM table_name ORDER BY column ASC/DESC;	SELECT * FROM Student ORDER BY cgpa DESC;
DISTINCT	Returns unique values	SELECT DISTINCT column_name FROM table_name;	SELECT DISTINCT branch FROM Student;
LIMIT	Limits the number of records returned	SELECT * FROM table_name LIMIT n;	SELECT * FROM Student LIMIT 5;
COUNT()	Counts the number of records	SELECT COUNT(*) FROM table_name;	SELECT COUNT(*) FROM Student;

SQLite3 Commands Used in Python:

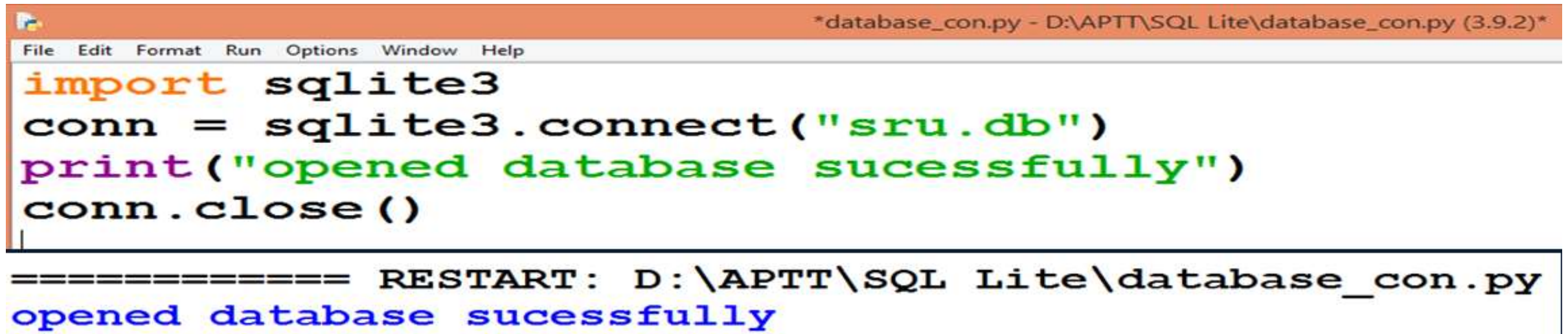
SQL Command	Purpose	Syntax	Example
MAX()	Returns the maximum value	SELECT MAX(column_name) FROM table_name;	SELECT MAX(cgpa) FROM Student;
MIN()	Returns the minimum value	SELECT MIN(column_name) FROM table_name;	SELECT MIN(cgpa) FROM Student;
AVG()	Calculates the average value	SELECT AVG(column_name) FROM table_name;	SELECT AVG(cgpa) FROM Student;
SUM()	Calculates the total of a numeric column	SELECT SUM(column_name) FROM table_name;	SELECT SUM(marks) FROM Student;
GROUP BY	Groups records based on a column	SELECT column_name, COUNT(*) FROM table_name GROUP BY column_name;	SELECT branch, COUNT(*) FROM Student GROUP BY branch;
HAVING	Filters grouped records	SELECT column_name FROM table_name GROUP BY column_name HAVING condition;	SELECT branch, COUNT(*) FROM Student GROUP BY branch HAVING COUNT(*) > 2;

CRUD Operations in SQLite3:

Operation	SQL Command	Example	Operation
Create	INSERT INTO	INSERT INTO Student(name, cgpa) VALUES('Anjali', 9.10);	Create
Read	SELECT	SELECT * FROM Student;	Read
Update	UPDATE	UPDATE Student SET cgpa=9.30 WHERE id=1;	Update
Delete	DELETE	DELETE FROM Student WHERE id=1;	Delete

Connect To Database

- Following Python code shows how to connect to an existing database.
- If the database does not exist, then it will be created and finally a database object will be returned.



```
*database_con.py - D:\APTT\SQL Lite\database_con.py (3.9.2)*
File Edit Format Run Options Window Help
import sqlite3
conn = sqlite3.connect("sru.db")
print("opened database sucessfully")
conn.close()

===== RESTART: D:\APTT\SQL Lite\database_con.py
opened database sucessfully
```

• Here, you can also supply database name as the special name **:memory:** to create a database in RAM. Now, let's run the above program to create our database **sru.db** in the current directory. You can change your path as per your requirement. Keep the above code in database.py file and execute it as shown below. If the database is successfully created, then it will display the following message.

CRUD (Create, Read, Update, Delete)

Creating a Table

Following Python program will be used to create a table in the previously created database.

```
import sqlite3

conn = sqlite3.connect('sru.db')
print("Opened database successfully")

conn.execute('''CREATE TABLE COMPANY
              (ID INT PRIMARY KEY     NOT NULL,
               NAME           TEXT     NOT NULL,
               AGE            INT      NOT NULL,
               ADDRESS        CHAR(50),
               SALARY         REAL);''')
print("Table created successfully")

conn.close()
```

```
===== RESTART: D:/APTT/SQL Lite/createtable.py
Opened database successfully
Table created successfully
```


CRUD (Create, Read, Update, Delete)

INSERT Operation

Following Python program shows how to create records in the COMPANY table created in the above example.

```
import sqlite3
conn = sqlite3.connect('sru.db')
print("Opened database successfully")

conn.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (1, 'Paul', 32, 'California', 20000.00 );")
conn.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (2, 'Allen', 25, 'Texas', 15000.00 );")
conn.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (3, 'Teddy', 23, 'Norway', 20000.00 );")
conn.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (4, 'Mark', 25, 'Rich-Mond ', 65000.00 );")

conn.commit()
print("Records created successfully")
conn.close()
```

```
===== RESTART: D:/APTT/SQL Lite/inserttable.py
Opened database successfully
Records created successfully
```


CRUD (Create, Read, Update, Delete)

SELECT Operation

Following Python program shows how to fetch and display records from the COMPANY table created in the above example.

```
import sqlite3

conn = sqlite3.connect('sru.db')
print("Opened database successfully")

cursor = conn.execute("SELECT id, name, address, salary from COMPANY")
for row in cursor:
    print("ID = ", row[0])
    print("NAME = ", row[1])
    print("ADDRESS = ", row[2])
    print("SALARY = ", row[3], "\n")

print("Operation done successfully")
conn.close()
```

```
Opened database successfully
ID = 1
NAME = Paul
ADDRESS = California
SALARY = 20000.0

ID = 2
NAME = Allen
ADDRESS = Texas
SALARY = 15000.0

ID = 3
NAME = Teddy
ADDRESS = Norway
SALARY = 20000.0

ID = 4
NAME = Mark
ADDRESS = Rich-Mond
SALARY = 65000.0

Operation done successfully
```

CRUD (Create, Read, Update, Delete)

UPDATE Operation

Following Python code shows how to use UPDATE statement to update any record and then fetch and display the updated records from the COMPANY table.

```
import sqlite3

conn = sqlite3.connect('sru.db')
print("Opened database successfully")

conn.execute("UPDATE COMPANY set SALARY = 25000.00 where ID = 1")
conn.commit()
print("Total number of rows updated :", conn.total_changes)

cursor = conn.execute("SELECT id, name, address, salary from COMPANY")
for row in cursor:
    print("ID = ", row[0])
    print("NAME = ", row[1])
    print("ADDRESS = ", row[2])
    print("SALARY = ", row[3], "\n")

print("Operation done successfully")
conn.close()
```

```
Opened database successfully
Total number of rows updated : 1
ID = 1
NAME = Paul
ADDRESS = California
SALARY = 25000.0

ID = 2
NAME = Allen
ADDRESS = Texas
SALARY = 15000.0

ID = 3
NAME = Teddy
ADDRESS = Norway
SALARY = 20000.0

ID = 4
NAME = Mark
ADDRESS = Rich-Mond
SALARY = 65000.0

Operation done successfully
```

CRUD (Create, Read, Update, Delete)

DELETE Operation

Following Python code shows how to use DELETE statement to delete any record and then fetch and display the remaining records from the COMPANY table.

```
import sqlite3

conn = sqlite3.connect('sru.db')
print("Opened database successfully")

conn.execute("DELETE from COMPANY where ID = 2;")
conn.commit()
print("Total number of rows deleted :", conn.total_changes)

cursor = conn.execute("SELECT id, name, address, salary from COMPANY")
for row in cursor:
    print("ID = ", row[0])
    print("NAME = ", row[1])
    print("ADDRESS = ", row[2])
    print("SALARY = ", row[3], "\n")

print("Operation done successfully")
conn.close()
```

```
Opened database successfully
Total number of rows deleted : 1
ID = 1
NAME = Paul
ADDRESS = California
SALARY = 25000.0

ID = 3
NAME = Teddy
ADDRESS = Norway
SALARY = 20000.0

ID = 4
NAME = Mark
ADDRESS = Rich-Mond
SALARY = 65000.0

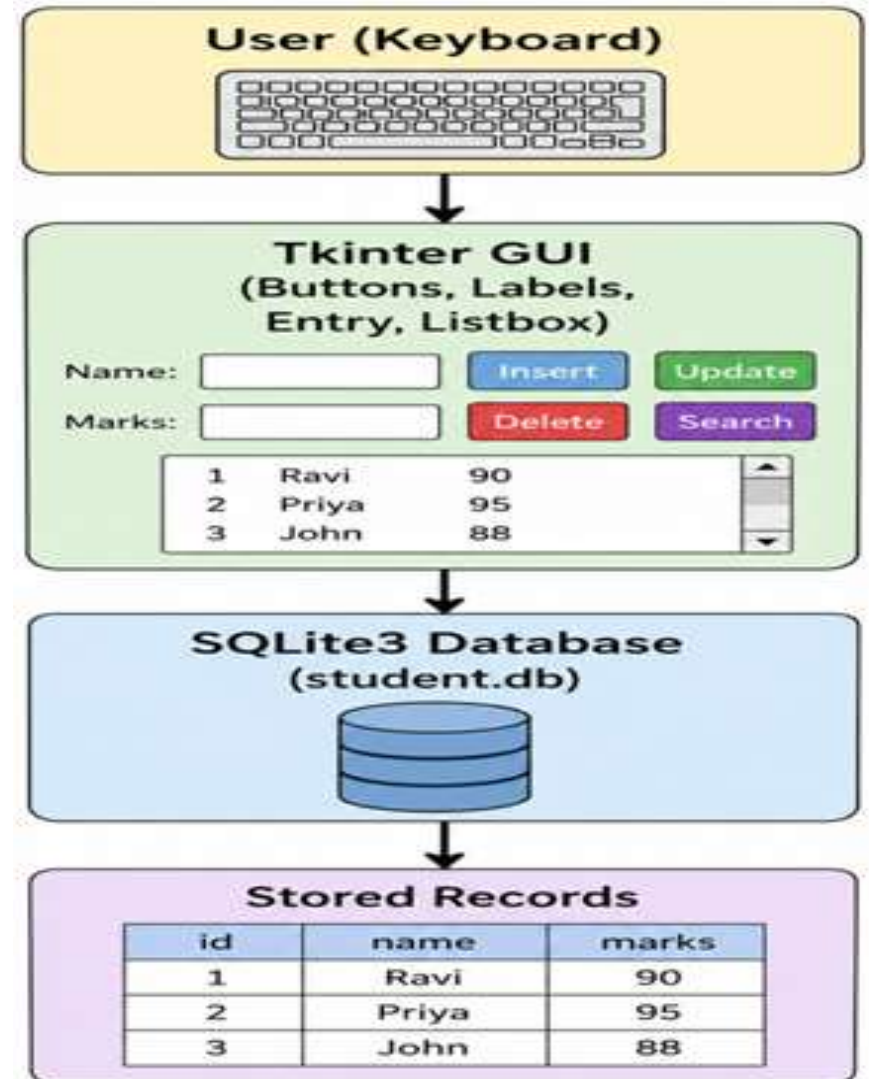
Operation done successfully
```


What is GUI–Database Integration?

GUI–Database Integration is the process of connecting a **Graphical User Interface (GUI)** with a **Database** so that users can store, retrieve, update, and delete data through graphical controls instead of typing SQL commands.

Why GUI with Database?

- Easy to use
- User-friendly
- No SQL knowledge required
- Faster data entry
- Efficient record management
- Reduces manual errors



```
from tkinter import *
from tkinter import messagebox
import sqlite3

# -----
# Database
# -----

conn = sqlite3.connect("student.db")
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS student(
id INTEGER PRIMARY KEY,
name TEXT,
marks INTEGER)
""")

conn.commit()
conn.close()
```

```
# -----
# Functions
# -----

def insert():
    try:
        conn = sqlite3.connect("student.db")
        cur = conn.cursor()

        cur.execute(
            "INSERT INTO student VALUES(?,?,?)",
            (
                int(id_entry.get()),
                name_entry.get(),
                int(marks_entry.get())
            )
        )

        conn.commit()
        conn.close()

        messagebox.showinfo("Success", "Record Inserted")
        display()

    except Exception as e:
        messagebox.showerror("Error", e)
```

Tkinter Widgets Used

Widget	Purpose
Label	Display text
Entry	User input
Button	Perform operations
Listbox	Display records
Frame	Group widgets
Scrollbar	Scroll records
MessageBox	Display messages

SQLite3 Functions Used

Function	Description
connect()	Connects to database
cursor()	Creates cursor
execute()	Executes SQL
commit()	Saves changes
fetchall()	Returns all records
fetchone()	Returns one record
close()	Closes connection


```
def display():
```

```
    listbox.delete(0,END)
    conn = sqlite3.connect("student.db")
    cur = conn.cursor()
    cur.execute("SELECT * FROM student")
    rows = cur.fetchall()
    for row in rows:
        listbox.insert(END,row)
    conn.close()
```

```
def update():
```

```
    conn = sqlite3.connect("student.db")
    cur = conn.cursor()
    cur.execute(
        "UPDATE student SET name=?,marks=? WHERE id=?",
        (
            name_entry.get(),
            marks_entry.get(),
            id_entry.get()
        )
    )
    conn.commit()
    conn.close()
    messagebox.showinfo("Updated","Record Updated")
    display()
```

```
def delete():
```

```
    conn = sqlite3.connect("student.db")
    cur = conn.cursor()

    cur.execute(
        "DELETE FROM student WHERE id=?",
        (id_entry.get(),)
    )
    conn.commit()
    conn.close()
    messagebox.showinfo("Deleted","Record Deleted")
    display()
```

```
def search():
```

```
    listbox.delete(0,END)
    conn = sqlite3.connect("student.db")
    cur = conn.cursor()
    cur.execute(
        "SELECT * FROM student WHERE id=?",
        (id_entry.get(),)
    )
    row = cur.fetchone()
    if row:
        listbox.insert(END,row)
    else:
        messagebox.showinfo("Search","Record Not Found")
    conn.close()
```

```
# -----  
# GUI  
# -----  
root = Tk()  
root.title("Student Database")  
root.geometry("500x500")  
Label(root, text="ID").pack()  
id_entry = Entry(root)  
id_entry.pack()  
Label(root, text="Name").pack()  
name_entry = Entry(root)  
name_entry.pack()  
Label(root, text="Marks").pack()  
marks_entry = Entry(root)  
marks_entry.pack()  
Button(root, text="Insert", command=insert, width=15).pack(pady=3)  
Button(root, text="Display", command=display, width=15).pack(pady=3)  
Button(root, text="Update", command=update, width=15).pack(pady=3)  
Button(root, text="Delete", command=delete, width=15).pack(pady=3)  
Button(root, text="Search", command=search, width=15).pack(pady=3)  
listbox = Listbox(root, width=50, height=12)  
listbox.pack(pady=10)  
display()
```

Student Database

ID
3013

Name
Nagendar Yamsani

Marks
98

Insert

Display

Update

Delete

Search

3013 {Nagendar Yamsani} 98
3022 {Vijay Kumar} 85
3076 {G. Ranjith Kumar} 95



Thank you